

Software Architecture Document (SAD) v1.0 – Sistema VetCare

Integrantes:

- Luis David Cruz Llica	2024-119060
- Jose Antonio Vilcanqui Chambi	2024-119017
- Cristian Chura Peralta	2017-119049
- Alexis Erik Ricardo Condori Rivera	2023-119015

SECCIÓN 1 – INTRODUCCIÓN

1.1 Propósito del documento

Describe que este SAD documenta las decisiones arquitectónicas del sistema de gestión clínica veterinaria VetCare, abarcando las vistas C4 (L1, L2, L3), decisiones de arquitectura (ADRs) y la especificación de la API.

1.2 Alcance

El sistema gestiona: agendamiento de citas, registro de mascotas, historial clínico, notificaciones y pagos. Atiende a los actores: Cliente (dueño), Recepcionista, Veterinario, Asistente veterinario y Administradora.

1.3 Definiciones y lenguaje ubicuo

Aquí listas el glosario que ya tienes en los bounded contexts:

- Cita: reserva de atención en fecha y hora determinada
- Diagnóstico (Dx): resultado clínico de una consulta
- Recordatorio: notificación automática previa a una cita
- Transacción: registro financiero de un pago procesado
- Mascota: animal registrado bajo un cliente dueño

SECCIÓN 2 – REPRESENTACIÓN ARQUITECTÓNICA

El sistema usa un modelo C4 en el cual representamos el modelo en tres niveles de detalle usando diagramas de secuencia en UML.

SECCIÓN 3 – MODELOS C4 UNIFICADOS

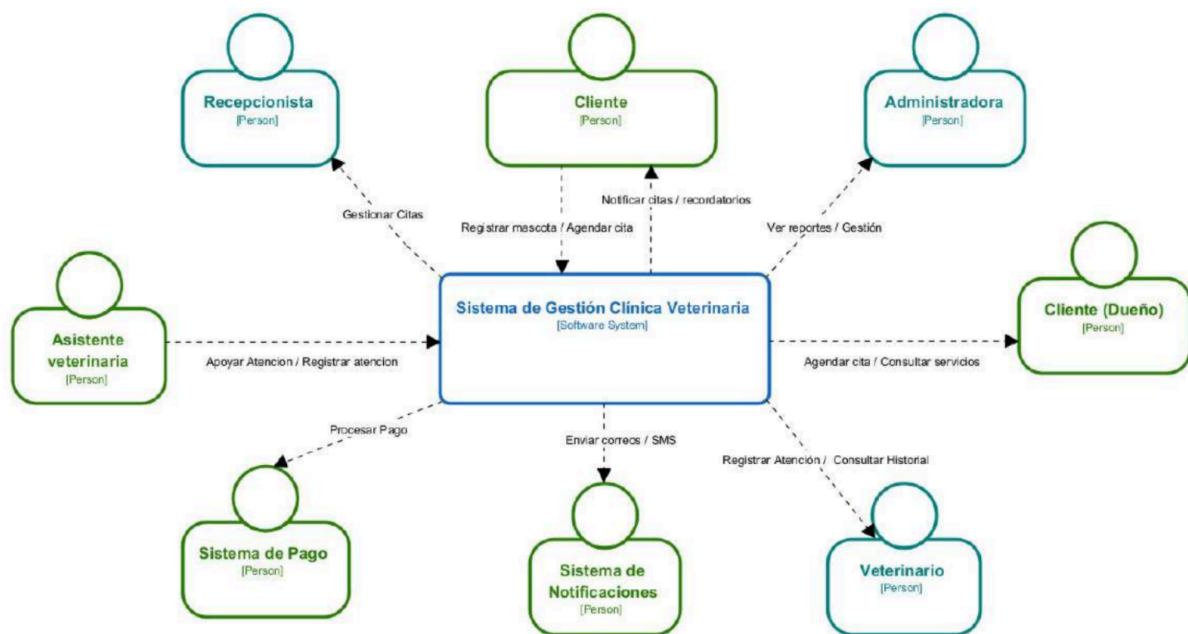
3.1 Nivel 1 – Diagrama de Contexto (L1)

El sistema central es el Sistema de Gestión Clínica Veterinaria. Interactúa con los siguientes actores y sistemas externos:

Actor / Sistema		Tipo	Interacción
Recepcionista		Person	Gestionar citas
Cliente		Person	Notificar citas / Registrar mascota / Agendar cita
Administradora		Person	Ver reportes / Gestión

Asistente Veterinaria	Person	Apoyar atención / Registrar atención
Cliente (Dueño)	Person	Agendar cita / Consultar servicios
Veterinario	Person	Registrar atención / Consultar historial
Sistema de Pago	Software System externo	Procesar pago
Sistema de Notificaciones	Software System externo	Enviar correo / SMS

Imagen 01:



3.2 Nivel 2 – Diagrama de Contenedores (L2)

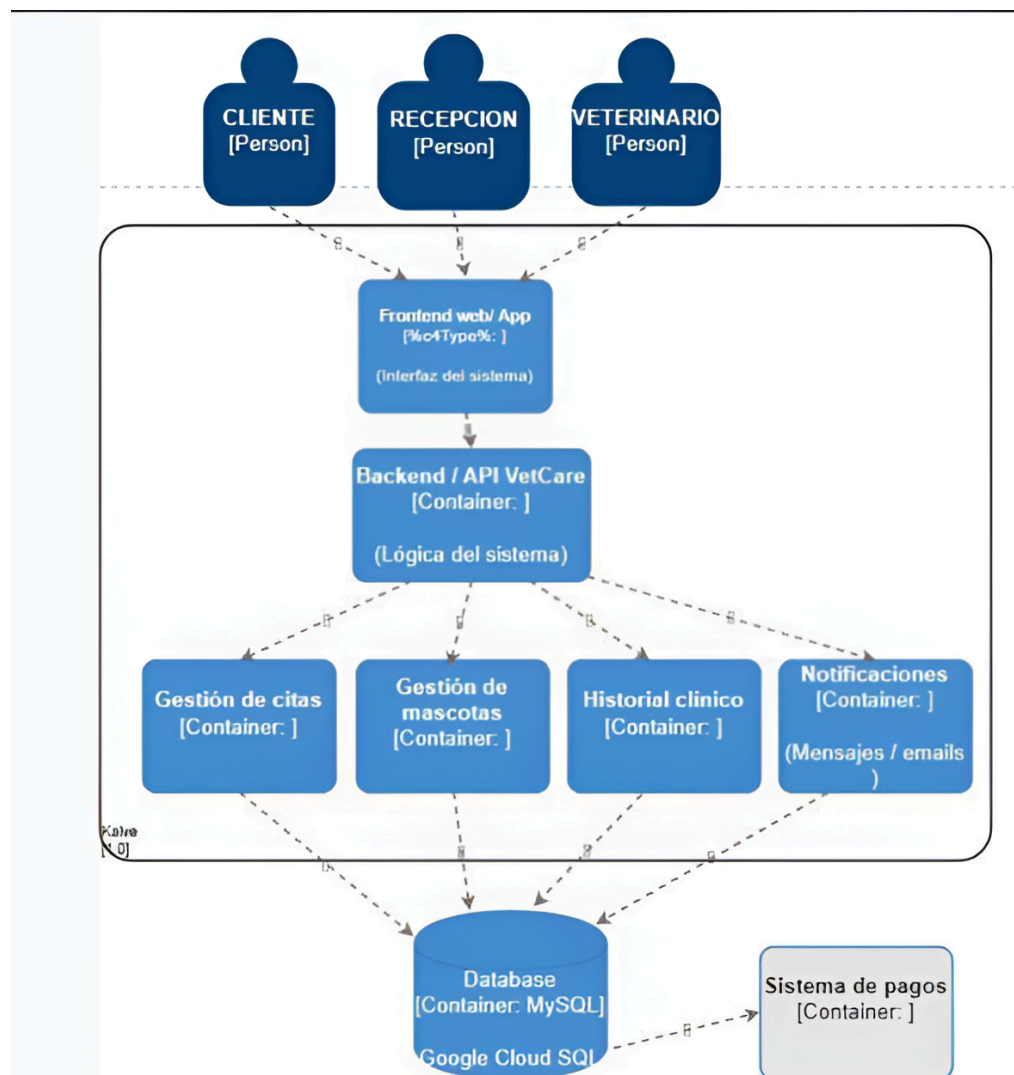
El sistema se descompone en los siguientes contenedores:

Contenedor	Tecnología	Responsabilidad
Frontend Web/App	Container	Lógica del sistema y orquestación
Backend / API VetCare	Container	Lógica del sistema y orquestación
Gestión de Citas	Container	Manejo de agenda y disponibilidad
Gestión de Mascotas	Container	Datos de animales registrados
Historial Clínico	Container	Información clínica por

		mascota
Notificaciones	Container	Mensajes / emails / SMS
Base de Datos	MySQL / Google Cloud SQL	Persistencia centralizada
Sistema de Pagos	Container externo	Procesamiento financiero

MADR: La separación en contenedores permite escalabilidad independiente por módulo, mantenibilidad aislada y separación de responsabilidades. Cada contenedor puede modificarse sin afectar a los demás.

Imagen 02:



3.3 Nivel 3 – Diagrama de Componentes (L3) – Backend/API VetCare

El backend se descompone en los siguientes componentes internos:

Componentes de aplicación:

- **AppointmentController** – punto de entrada del frontend al backend

- **AppointmentFacade** – orquesta el caso de uso "Crear cita + registrar pago" (patrón GoF Facade)
- **AppointmentService** – lógica de registro y persistencia de citas
- **AvailabilityService** – verifica disponibilidad de horarios
- **PetService** – valida datos de la mascota seleccionada
- **PaymentService** – gestiona cobro e interacción con pasarela externa
- **NotificationService** – programa recordatorios y confirmaciones

Repositorios (persistencia en MySQL):

- AppointmentRepository
- PetRepository
- PaymentRepository

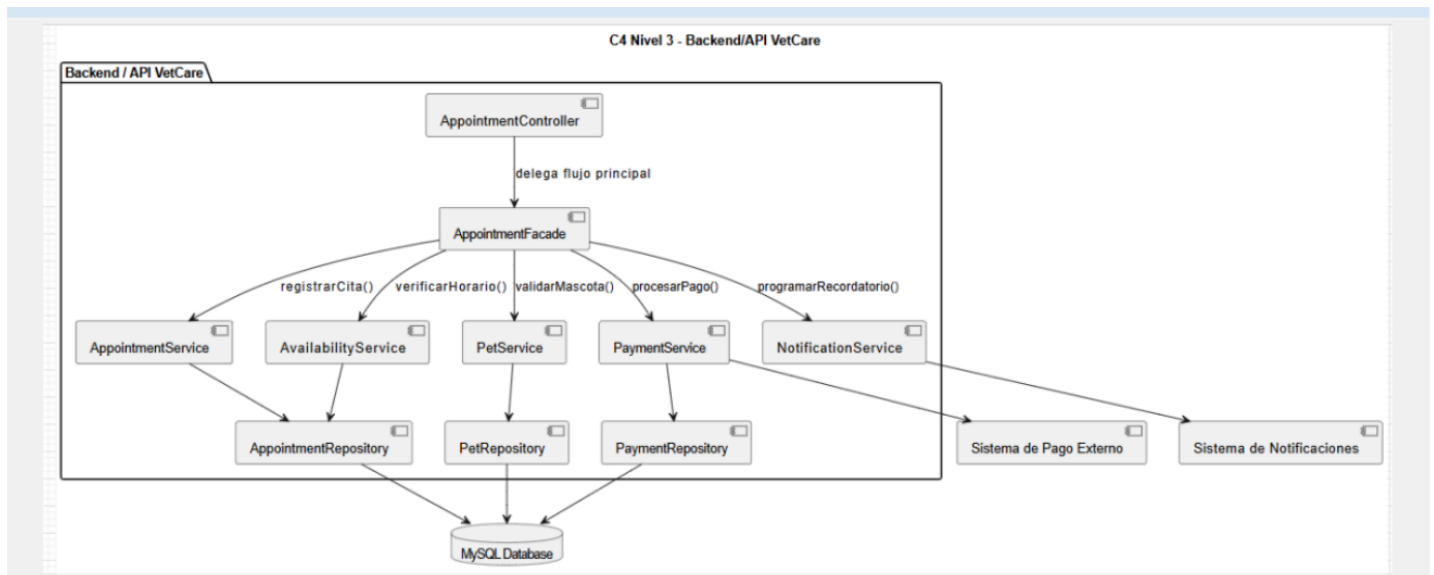
Sistemas externos integrados:

- Sistema de Pago Externo
- Sistema de Notificaciones

Para el módulo de Atención Médica (RF-006), componentes adicionales:

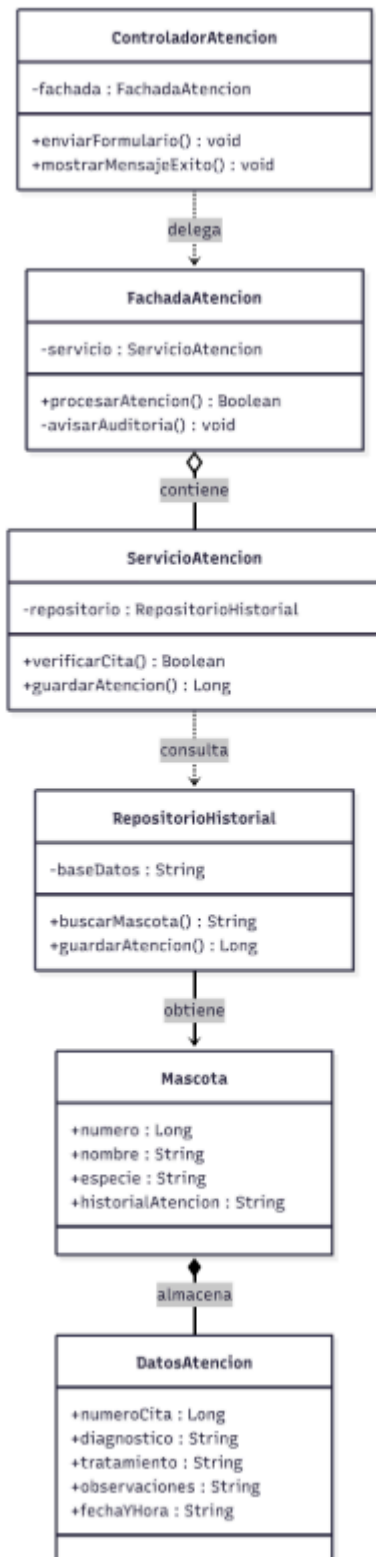
- **ControladorAtencion** → delega a **FachadaAtencion**
- **FachadaAtencion** → coordina **ServicioAtencion**
- **ServicioAtencion** → consulta **RepositorioHistorial**
- **RepositorioHistorial** → obtiene **Mascota** → almacena **DatosAtencion**

Imagen 03:



SECCIÓN 4 – VISTAS DINÁMICAS

1. DIAGRAMA DE CLASES



2. DIAGRAMA DE SECUENCIA UML

Caso de uso crítico: "Crear cita + registrar pago"

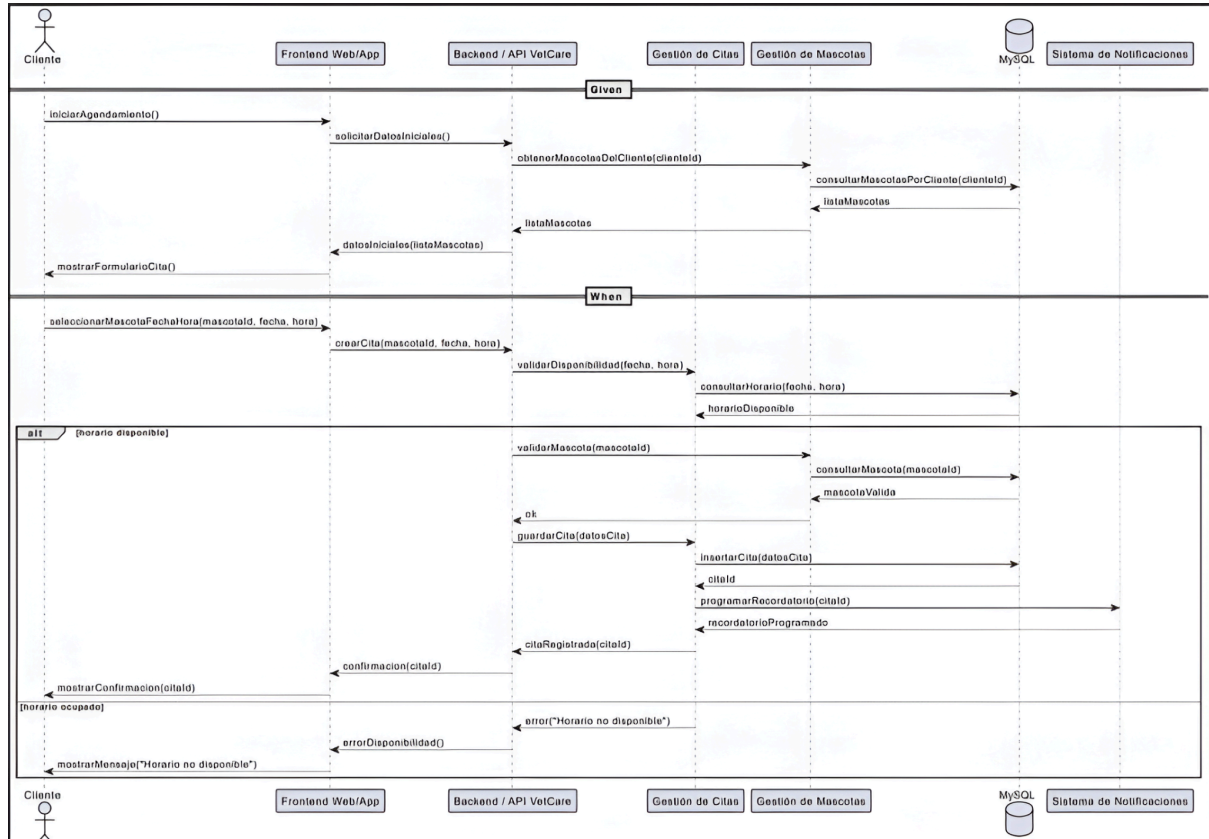
Trazabilidad Given / When / Then:

- Given: Cliente con sesión activa, mascota registrada, horarios disponibles, cargos configurados
- When: Cliente selecciona mascota, fecha/hora y confirma pago

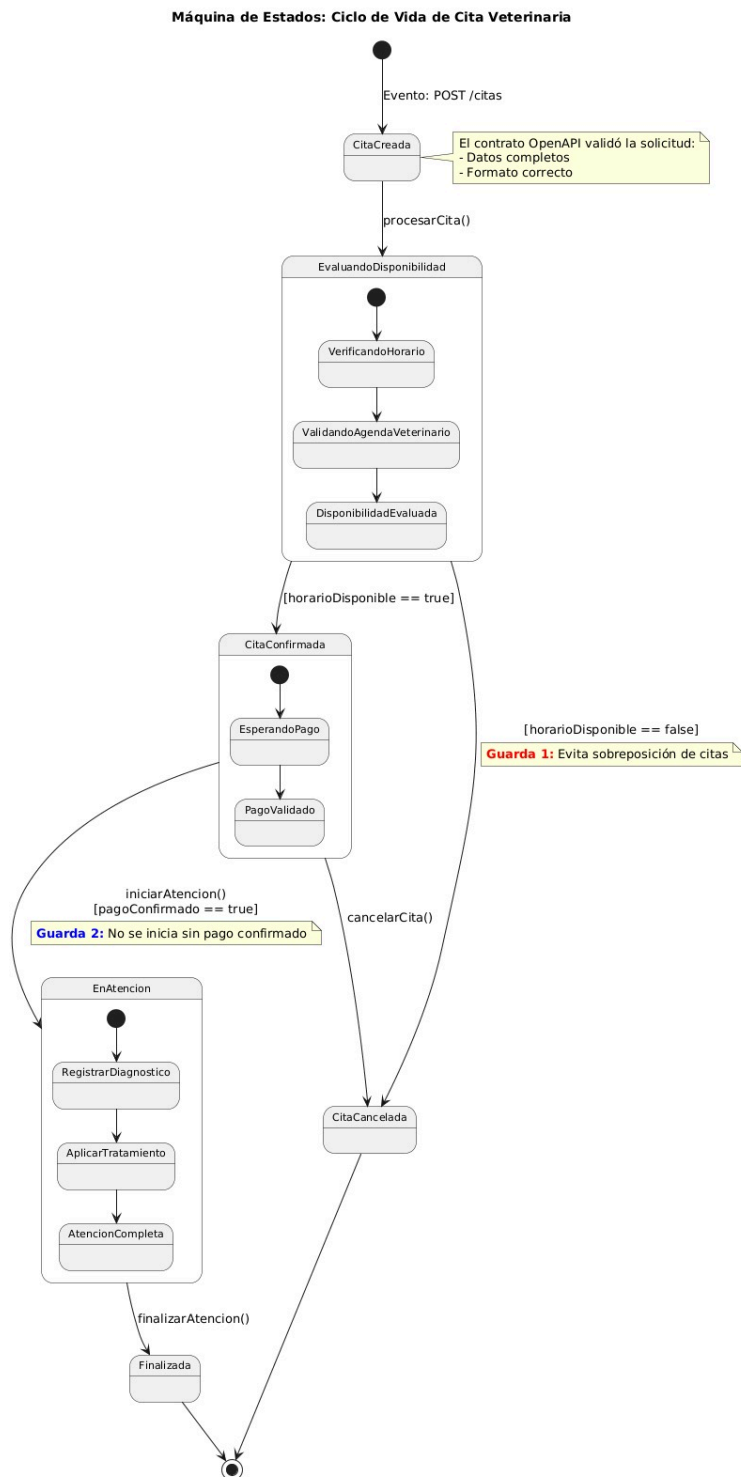
- Then: Sistema valida disponibilidad, valida mascota, registra cita, procesa pago, almacena transacción, programa notificación, muestra confirmación o error

Flujos alternos documentados:

1. Horario no disponible → backend detiene flujo → frontend muestra mensaje de error de disponibilidad
2. Pago rechazado → sistema registra intento fallido en BD → frontend muestra mensaje de error de pago



2. MÁQUINA DE ESTADOS-Ciclo de vida

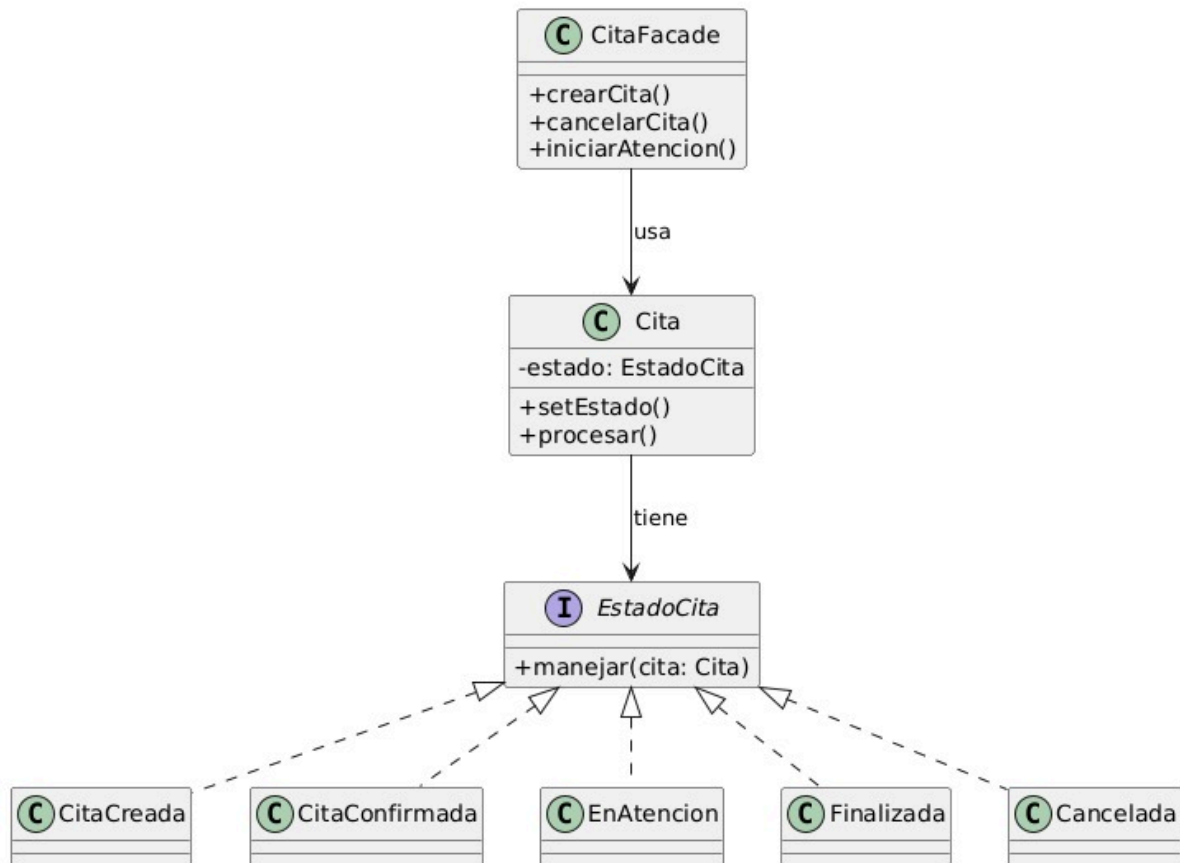


El ciclo de vida de la entidad Cita inicia con el evento POST / citas, validado por el contrato OpenAPI. Desde el estado CitaCreada se evalúa la disponibilidad mediante el estado compuesto EvaluandoDisponibilidad. La primera guarda **[horarioDisponible == true]** permite avanzar a CitaConfirmada; si es falsa, deriva a Cancelada. En CitaConfirmada, la segunda guarda **[pagoConfirmado == true]** habilita la transición a EnAtencion. Finalmente, tras completar diagnóstico y tratamiento, la cita alcanza el

estado Finalizada. En cualquier etapa previa a EnAtencion puede invocarse cancelarCita().

3. Diseño Táctico: Facade y State

Diseño Táctico: Facade + State (Gestión de Citas)



CitaFacade actúa como punto de entrada único, exponiendo las operaciones crearCita(), cancelarCita() e iniciarAtencion(). La entidad Cita delega su comportamiento en la interfaz EstadoCita, implementada por las clases CitaCreada, CitaConfirmada, EnAtencion, Finalizada y Cancelada. Cada clase encapsula su propia lógica de transición, eliminando condicionales complejos y garantizando que las reglas de negocio de la máquina de estados se respeten en la implementación.

SECCIÓN 5 – CONTRATO OPENAPI v3 CONSOLIDADO

Esta sección probablemente es la que más necesitas construir. Basándote en todos los flujos y componentes documentados, el contrato debe incluir estos endpoints mínimos:

```
openapi: 3.2.0
info:
  title: VetCare - Servicio de Gestion de Citas y Atencion
  description: |
```


Contrato de frontera API-first para la gestion de citas y atenciones veterinarias.

Permite registrar citas, gestionar atenciones clinicas y consultar historial.

version: 1.0.0

servers:

- url: /api/v1
- description: Servidor principal

paths:

/citas:

get:

tags: [Citas]

summary: Listar citas

operationId: listarCitas

responses:

"200":

description: Lista de citas

content:

application/json:

schema:

type: array

items:

\$ref: "#/components/schemas/CitaResponse"

post:

tags: [Citas]

summary: Crear una nueva cita

operationId: crearCita

requestBody:

required: true

content:

application/json:

schema:

\$ref: "#/components/schemas/CitaRequest"

responses:

"201":

description: Cita creada correctamente

content:

application/json:

schema:

```
        $ref: "#/components/schemas/CitaResponse"
    "400":
        description: Datos inválidos
        content:
            application/json:
                schema:
                    $ref: "#/components/schemas/Error"

/citas/{citaId}:
    get:
        tags: [Citas]
        summary: Obtener una cita por ID
        operationId: obtenerCita
        parameters:
            - $ref: "#/components/parameters/CitaId"
        responses:
            "200":
                description: Detalle de la cita
                content:
                    application/json:
                        schema:
                            $ref: "#/components/schemas/CitaResponse"
            "404":
                description: Cita no encontrada
                content:
                    application/json:
                        schema:
                            $ref: "#/components/schemas/Error"

    delete:
        tags: [Citas]
        summary: Cancelar una cita
        operationId: cancelarCita
        parameters:
            - $ref: "#/components/parameters/CitaId"
        responses:
            "200":
                description: Cita cancelada
                content:
                    application/json:
                        schema:
                            type: object
```

```
/atenciones:
  post:
    tags: [Atencion]
    summary: Registrar una atencion medica
    operationId: registrarAtencion
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/AtencionRequest"
    responses:
      "201":
        description: Atencion registrada correctamente
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/AtencionResponse"
```

```
/atenciones/{atencionId}:
  get:
    tags: [Atencion]
    summary: Obtener atencion por ID
    operationId: obtenerAtencion
    parameters:
      - $ref: "#/components/parameters/AtencionId"
    responses:
      "200":
        description: Detalle de la atencion
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/AtencionResponse"
```

```
components:
  schemas:
    BaseCita:
      type: object
      required:
        - clienteId
        - fecha
```

```
- hora
- mascotaId
properties:
  mascotaId:
    type: string
  clienteId:
    type: string
  fecha:
    type: string
    format: date
  hora:
    type: string
  motivo:
    type: string

CitaRequest:
  type: object
  allOf:
    - $ref: "#/components/schemas/BaseCita"
    - type: object
      properties:
        prioridad:
          type: string
          enum: [normal, urgente]

CitaResponse:
  type: object
  allOf:
    - $ref: "#/components/schemas/BaseCita"
    - type: object
      properties:
        citaId:
          type: string
        estado:
          type: string
          enum: [creada, confirmada, en_proceso, finalizada, cancelada]

AtencionBase:
  type: object
  required:
    - fecha
    - mascotaId
```

```
- motivo
- veterinarioId
properties:
  mascotaId:
    type: string
  veterinarioId:
    type: string
  fecha:
    type: string
    format: date
  motivo:
    type: string

AtencionRequest:
  type: object
  allOf:
    - $ref: "#/components/schemas/AtencionBase"
    - type: object
      properties:
        diagnostico:
          type: string
        tratamiento:
          $ref: "#/components/schemas/Tratamiento"

AtencionResponse:
  type: object
  allOf:
    - $ref: "#/components/schemas/AtencionRequest"
    - type: object
      properties:
        atencionId:
          type: string
        estado:
          type: string
          enum: [registrada, en_revision, completada, anulada]

Tratamiento:
  type: object
  properties:
    descripcion:
      type: string
    indicaciones:
```

```

    type: string

Error:
  type: object
  properties:
    codigo:
      type: string
    mensaje:
      type: string

parameters:
  CitaId:
    name: citaId
    in: path
    required: true
    schema:
      type: string

  AtencionId:
    name: atencionId
    in: path
    required: true
    schema:
      type: string

```

SECCIÓN 6 – MATRIZ DE TRAZABILIDAD

Esta tabla conecta requisitos → componentes → endpoints → ADRs:

Requisito	Descripción	Componente C4 L3	Endpoint / OperationId	Persistencia	ADR relacionado
RF-001	Agendar cita	AppointmentFacade, CitaFacade, AvailabilityService	POST /citas – <code>crearCita</code>	RepositorioCita / t_cita	ADR-001, ADR-002, ADR-003
RF-002	Cancelar cita	AppointmentService, CitaFacade	DELETE /citas/{citaId} – <code>cancelarCita</code>	RepositorioCita / t_cita	ADR-001, ADR-003
RF-003	Consultar / reprogramar cita	AppointmentService	GET /citas/{citaId} – <code>obtenerCita</code>	RepositorioCita / t_cita	ADR-001, ADR-003

RF-004	Registrar mascota	PetService, PetRepository	POST /mascotas	PetRepository / t_mascota	ADR-001
RF-005	Consultar mascotas del cliente	PetService	GET /mascotas	PetRepository / t_mascota	ADR-001, ADR-003
RF-006	Registrar atención médica	FachadaAtencion, ServicioAtencion, RepositorioHistorial	POST /atenciones – registrarAtencion / GET /atenciones/{atencionId} – obtenerAtencion	RepositorioHistorial / t_cita	ADR-001, ADR-002, ADR-003
RF-007	Enviar notificaciones	NotificationService	POST /notificaciones	— (sistema externo)	ADR-001
RF-008	Procesar pago	PaymentService, PaymentRepository	POST /pagos	PaymentRepository	ADR-001, ADR-002

SECCIÓN 7 – ARCHITECTURAL DECISION RECORDS (ADRs)

ADR-001: Selección del Estilo Arquitectónico

Estado: Aprobado

Contexto: El sistema VetCare presenta múltiples dominios funcionales (citas, historial, notificaciones, pagos). El equipo es de 3 integrantes, lo que limita la viabilidad de arquitecturas altamente distribuidas. Se prioriza velocidad de implementación, comprensión del sistema y bajo costo de mantenimiento inicial.

Decisión: Arquitectura monolítica modular, donde todos los componentes se integran en una sola aplicación pero organizados internamente en módulos que representan los Bounded Contexts identificados.

Rationale: La arquitectura monolítica modular reduce la carga cognitiva del equipo (Greiler et al.), evita la gestión de múltiples servicios distribuidos, protocolos de comunicación y configuraciones separadas. Permite que el equipo se concentre en el dominio del negocio. Según ISO 25010, un sistema puede ser técnicamente mantenible pero generar alta carga cognitiva si su estructura es difícil de comprender; esta decisión mejora ambas dimensiones simultáneamente.

Consecuencias positivas: comprensión centralizada, despliegue simplificado, desarrollo más rápido para equipos pequeños, menor cantidad de tecnologías a manejar.

Consecuencias negativas: limitaciones de escalabilidad a largo plazo, un error puede propagarse entre módulos si no se controla la modularidad, dificultad de mantenimiento si crece sin disciplina modular.

ADR-002: Aplicación del Patrón Facade (GoF) en Módulos Críticos

Estado: Aprobado

Contexto: Durante el diseño del caso de uso "Crear cita + registrar pago" (RF-001, RF-008) y del módulo de Registro de Atención Médica (RF-006), se identificó que los controladores

debían interactuar directamente con múltiples servicios y repositorios simultáneamente (disponibilidad, mascotas, citas, pagos, notificaciones, persistencia). Esto genera alto acoplamiento y dificulta el mantenimiento.

Decisión: Aplicar el patrón Facade (Gamma et al., GoF 1995), introduciendo las clases AppointmentFacade y FachadaAtencion como únicos intermediarios entre cada controlador y los subsistemas internos.

Rationale: El patrón Facade encapsula la complejidad del caso de uso y ofrece una interfaz unificada al controlador. Se evaluaron tres opciones: (1) conexión directa controlador-servicios, rápida pero con acoplamiento alto; (2) patrón Facade, que reduce dependencias del controlador a una sola clase manteniendo responsabilidades claras — opción elegida; (3) patrón Mediator, también de GoF, descartado por resultar demasiado complejo para la cantidad de componentes del flujo actual. La decisión sigue el principio de alta cohesión y bajo acoplamiento del SWEBOK v4.0a y la separación de responsabilidades de Sommerville (2016).

Consecuencias positivas: menor acoplamiento entre componentes, mayor claridad del flujo principal, mejor organización de responsabilidades, facilidad de pruebas unitarias por módulo, escalabilidad del backend.

Consecuencias negativas: se agrega una clase extra que en proyectos pequeños puede parecer sobrediseño; hay que vigilar que la fachada no acumule demasiada lógica propia con el tiempo.

ADR-003: Patrón Repository con Clean Architecture para la Capa de Persistencia

Estado: Aprobado

Contexto:

El módulo de gestión de citas (RF-001, RF-002, RF-003) requiere un mecanismo de almacenamiento que permita guardar y recuperar información de citas veterinarias. El riesgo identificado es que, sin una capa intermedia, la lógica de negocio del dominio quedaría acoplada directamente a los detalles tecnológicos de MySQL, lo que dificultaría el mantenimiento y haría imposible cambiar la tecnología de almacenamiento sin modificar el dominio.

Decisión:

Se adoptó el patrón Repository siguiendo los principios de Clean Architecture (Martin, 2018). Se define la interfaz RepositorioCita en la capa de dominio, con tres operaciones expresadas en lenguaje de negocio: buscarPorId, guardar y eliminar. Su implementación concreta RepositorioCitaMySQL reside en la capa de infraestructura y es el único componente que conoce los detalles de MySQL.

Rationale:

Se evaluaron dos alternativas. La primera fue acceder directamente a la base de datos desde los servicios del dominio, lo que resulta rápido de implementar pero genera un acoplamiento alto que viola el principio de inversión de dependencias (DIP). La segunda, elegida, fue el patrón Repository con separación de capas, que mantiene la dependencia apuntando siempre hacia adentro: la infraestructura depende del dominio, nunca al revés. Esta decisión se fundamenta en Martin (2018) y en Fowler (2003), quienes establecen que el dominio debe simular una colección de objetos en memoria, ignorando completamente si la implementación usa MySQL, un archivo de texto o un servicio en la nube.

Consecuencias positivas:

- El dominio permanece independiente de cualquier tecnología de almacenamiento.

- Las excepciones técnicas de SQL son capturadas dentro de RepositorioCitaMySQL y traducidas a excepciones del dominio (CitaNoEncontradaException, CitaPersistenciaException), evitando que los detalles de infraestructura contaminen el círculo interno.
- La eliminación es lógica (cambio de estado a CANCELADO), preservando la integridad del historial clínico y cumpliendo los requisitos de auditoría.
- Cada componente puede probarse de forma aislada.

Consecuencias negativas:

- Se agrega una interfaz extra que en proyectos muy pequeños puede parecer sobrediseño.
- Requiere disciplina del equipo para no romper la regla de dependencias al agregar nuevas funcionalidades.

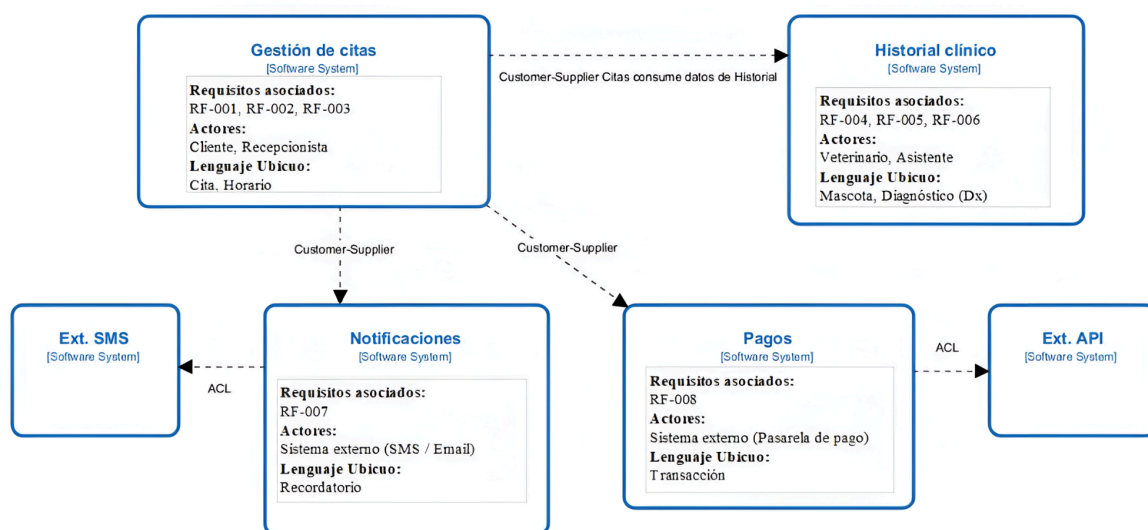
SECCIÓN 8 – BOUNDED CONTEXTS Y CONTEXT MAP

- Gestión de Citas (RF-001 a RF-003): actores Cliente y Recepcionista, lenguaje ubicuo Cita/Horario
- Historial Clínico (RF-004 a RF-006): actores Veterinario y Asistente, lenguaje ubicuo Mascota/Diagnóstico
- Notificaciones (RF-007): sistema externo SMS/Email, lenguaje ubicuo Recordatorio
- Pagos (RF-008): sistema externo Pasarela, lenguaje ubicuo Transacción

Relaciones entre contextos (notación Vernon):

- Gestión de Citas → Historial Clínico: Customer–Supplier
- Gestión de Citas → Notificaciones: Customer–Supplier + ACL hacia SMS/Email externo
- Gestión de Citas → Pagos: Customer–Supplier + ACL hacia API de pasarela externa

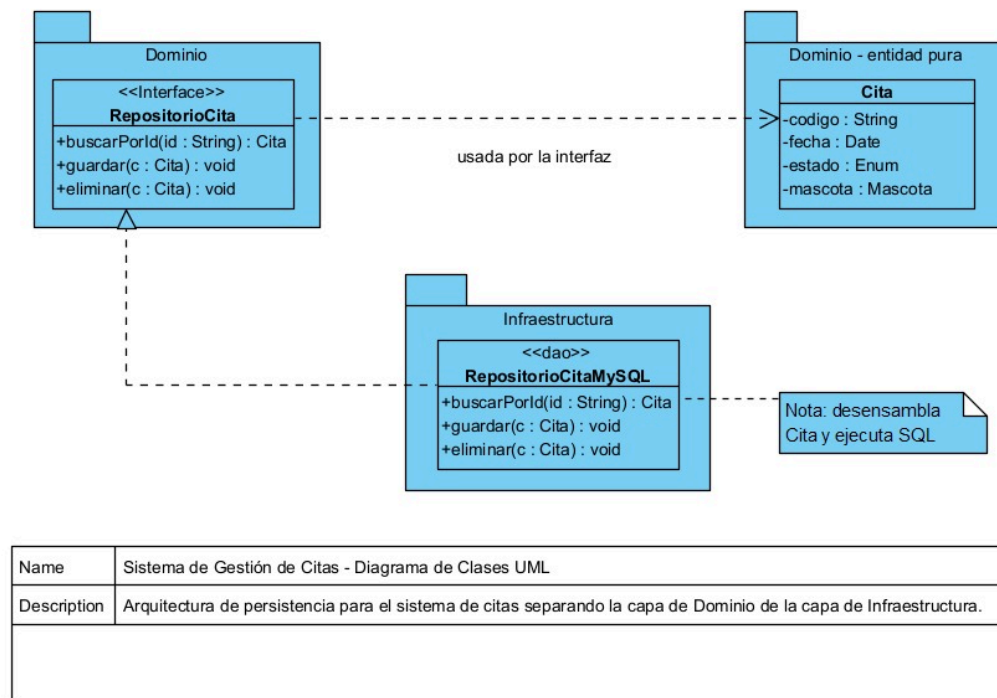
Diagrama de contextos:



SECCIÓN 9 – Diseño de Persistencia Desacoplada

9.1 Introducción.

El diseño de persistencia del sistema VetCare sigue los principios de Clean Architecture, desacoplando la capa de dominio de los detalles tecnológicos de almacenamiento. Se emplea el patrón Repository, que define un contrato abstracto para el acceso a datos y delega su implementación a la capa de infraestructura. Esto garantiza que el dominio permanezca independiente, facilitando la mantenibilidad, escalabilidad y adaptabilidad del sistema ante cambios en la tecnología de almacenamiento.



SECCIÓN 10 – CONCLUSIONES DEL SAD

1. El modelo C4 en tres niveles permite comunicar la arquitectura a distintas audiencias con el nivel de detalle apropiado para cada una.
2. La arquitectura monolítica modular fue la decisión correcta para el tamaño y capacidades del equipo, sin sacrificar la separación de responsabilidades mediante Bounded Contexts.
3. El patrón Facade redujo el acoplamiento en los flujos más críticos del sistema, mejorando mantenibilidad y testeabilidad.
4. La trazabilidad completa desde requisitos hasta endpoints y componentes garantiza que el diseño responde a todas las necesidades funcionales identificadas.

SECCIÓN 11 - ANEXOS

Configuración Arquitectónica en Figma

VetCare

TableroPacientesCitasInventario

Nueva Cita

1

Paso 1

Agendar Cita

2

Paso 2

Detalles Clínicos

3

Paso 3

Confirmación

Agendar Cita

Seleccione la mascota y el horario disponible.

Mascota Registrada

Luna (Perro - Golden Retriever)

▼▼

Fecha y Hora

<Octubre 2024>

Lu	Ma	Mi	Ju	Vi	Sa	Do
28	29	30	1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25

HORARIOS DISPONIBLES - 14 OCT

09:00 AM

09:30 AM

10:00 AM

10:30 AM

11:00 AM

11:30 AM

Continuar al Pago

Figura 1- Agendar Cita

Tablero

Pacientes

Citas

Inventario

Nueva Cita

Checkout

Complete el pago para confirmar la cita.

✓

Paso 1: Agendar Cita

2

Paso 2: Pago

3

Paso 3: Confirmación

Detalles de Pago

Titular de la tarjeta

Nombre como aparece en la tarjeta

Número de tarjeta

0000 0000 0000 0000

Fecha de expiración

MM/AA

CW

123

Resumen de la Cita

Mascota

Luna

Perro - Golden Retriever

Fecha

14 Octubre 2024

Hora

10:00 AM

Costo Total

S/.45.00

Confirmar Pago

Complete el formulario para habilitar el pago.

Figura 2 - Checkout

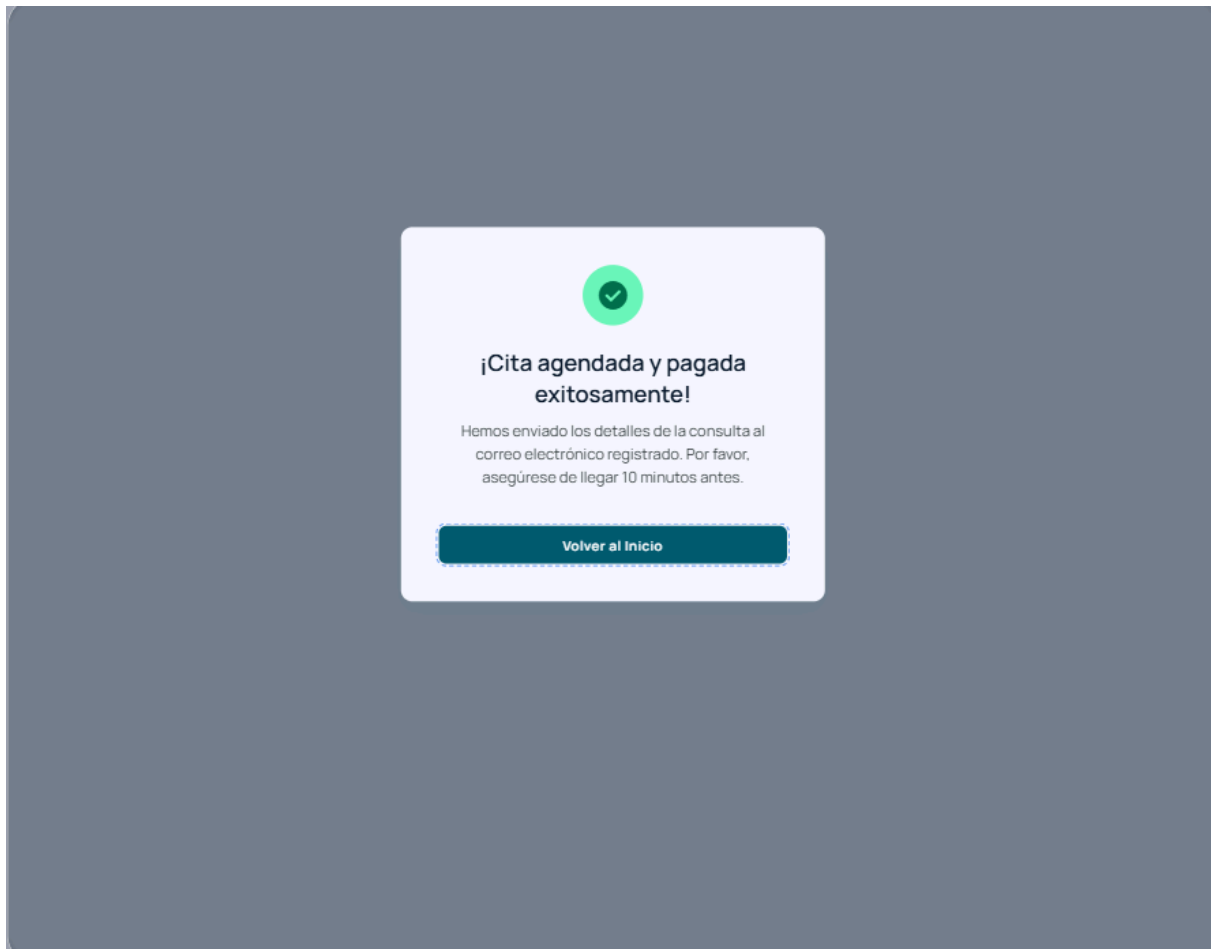


Figura 3 - Alerta de Pago Exitoso

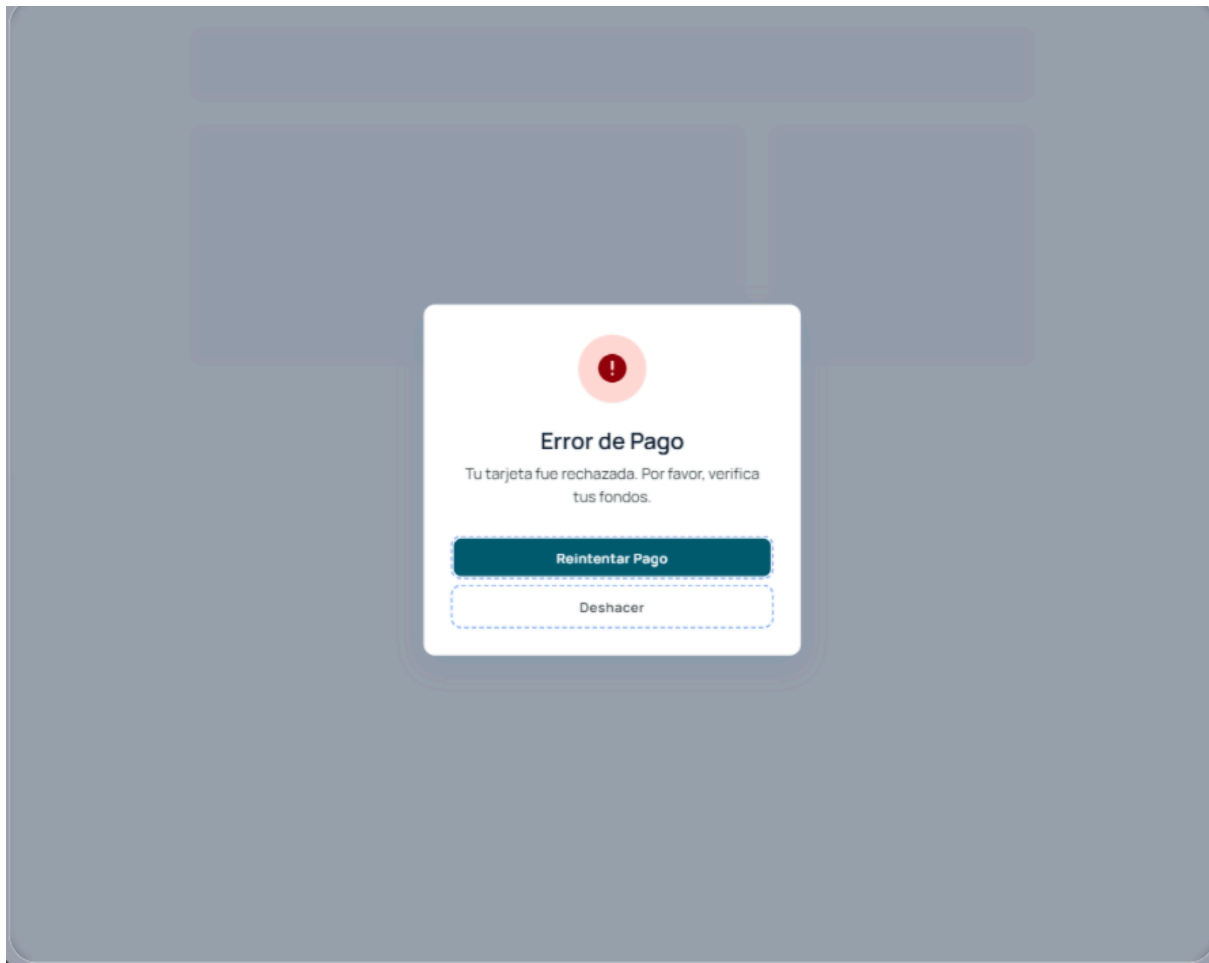


Figura 4 - Alerta de Error de Pago

Auditoría Heurística - Interfaz VetCare (HiFi)

1. Evaluación según las 10 Heurísticas de Nielsen El diseño del flujo "Crear cita + registrar pago" ha sido evaluado bajo los principios de Nielsen, evidenciando los siguientes cumplimientos en las pantallas desarrolladas:

- **Visibilidad del estado del sistema:** En las pantallas de *Agendar Cita* y *Checkout*, se implementó un indicador de progreso superior (stepper) que resalta claramente si el usuario se encuentra en el "Paso 1" o "Paso 2", proporcionando feedback constante sobre su ubicación en el flujo.
- **Reconocer en lugar de recordar:** En la pantalla de *Checkout* (Paso 2), se diseñó una tarjeta lateral denominada "Resumen de la Cita" que muestra la mascota seleccionada (Luna), la fecha (14 Octubre 2024), la hora y el costo total. Esto libera al usuario de la carga de mantener esta información en su memoria a corto plazo.
- **Prevención de errores:** En la pantalla del Paso 1, se restringió la entrada de datos mediante el uso de un componente *Drop-down* para la selección de la mascota y un calendario interactivo con botones bloqueados para los horarios, evitando errores de tipeo. En el Paso 2, el botón de pago cuenta con un candado visual y una instrucción clara que indica: *"Complete el formulario para habilitar el pago"*.
- **Control y libertad del usuario:** En el estado de error (Modal de Error de Pago), se habilitó claramente un botón secundario de "Deshacer", sirviendo como salida de

emergencia para el usuario en caso de que decida no reintentar la transacción inmediatamente.

2. Evaluación según las 8 Reglas de Oro de Shneiderman

- **Diseñar diálogos de cierre:** El flujo culmina exitosamente con un modal de confirmación claro (check verde) que indica "*¡Cita agendada y pagada exitosamente!*" y provee un botón para "Volver al Inicio", otorgando satisfacción de logro al operador.
- **Ofrecer retroalimentación informativa:** Para el escenario de fallo, el sistema responde con un modal específico de "Error de Pago", asegurando que el operador siempre sepa el resultado de su acción en el backend.

Justificación del Desafío Técnico: Prevención y Feedback

El módulo crítico transaccional asume la inminencia del error humano y las fallas de integración con sistemas externos (como la pasarela de pagos).

Para mitigar los fallos (*mistakes* y *slips*), el diseño de la interfaz actúa como una capa formal de defensa:

1. **Manejo de Errores Constructivo:** Si la transacción falla, el sistema no muestra un código de error genérico (ej. "Error 500") ni culpa al usuario. En su lugar, el modal de error indica claramente "*Tu tarjeta fue rechazada. Por favor, verifica tus fondos*", ofreciendo un botón primario para "Reintentar Pago" y una opción rápida para "Deshacer".
2. **Restricciones de Interfaz (Sensibility Checks):** El diseño protege la integridad de los datos que viajarán a la API. Al forzar selecciones estructuradas (menús desplegables para la mascota y botones discretos para las horas) y deshabilitar el botón de envío principal hasta que los formularios obligatorios estén llenos, se previenen llamadas inválidas a la base de datos MySQL subyacente.

Link de nuestro diseño en Stitch: [Click aqui](#)

SECCIÓN 12: LOG DEL WALKTHROUGH

Revisado por el equipo de:

Luciano Fabricio Vélez Miranda	2024-119025
Corimaya chura Stip Leonel yordy	2024-119032
Choquehuanca Rafael Ronald	2024-119019

ID	ARTEFACTO REVISADO	Observación / Defecto Detectado	Severidad	Acción Correctiva Propuesta	Responsable	Estado
1	C4 L2 / C4 L3	El diagrama C4 Nivel 2 muestra "Gestión de Citas" y "Gestión de Mascotas" como contenedores independientes, pero el diagrama Nivel 3 los agrupa como componentes internos de un único contenedor "Backend"	Alta	Decidir si son unidades de despliegue separadas (L2) o módulos lógicos (L3). Si son contenedores, el Nivel 3 debe ser un zoom individual de uno de ellos	Ronald Choquehuanca Rafael	Abierto
2	C4 L1 / Context Map	El Nivel 1 identifica a "Administradora" y "Asistente Veterinaria" como actores clave, sin embargo, el Context Map (notación Vernon) y la matriz de trazabilidad omiten sus interacciones y fronteras de significado	Media	Homologar los actores en todas las vistas basándose en el Lenguaje Ubicuo definido en la introducción del SAD	Ronald Choquehuanca Rafael	Abierto
3	OpenAPI v3.2.0	El contrato OpenAPI define citald y atencionId como simples string, ignorando la decisión técnica de usar UUID para garantizar identidades de negocio estables y no predecibles	Media	Actualizar los esquemas de parámetros y objetos en el YAML a type: string con format: uuid	Stip	Abierto
4	OpenAPI v3.2.0	El endpoint DELETE /citas/{citald} especifica una respuesta 200 con un type: object vacío, lo que incumple el estándar de retroalimentación informativa para el desarrollador (DX)	Baja	Definir un esquema de respuesta que confirme el nuevo estado de la entidad (ej. Estado: Cancelada) para facilitar la evaluación en el cliente	Luciano	Abierto
5	C4 L1 / Context Map	Identificación pobre en el esquema de contextos, con respecto a los sistemas externos son tomados como actores involucrados.	Baja	Definir bien cuales son los actores y cuales los sistemas externos definiendolos con un rectangulo de distinto color y no como una persona.	Stip	Abierto
6	Vista Lógica (Clases) / ADR	Falta de Trazabilidad en Clases: El Diagrama de Clases muestra a FachadaAtencion delegando a ServicioAtencion, pero no refleja los métodos de validación de disponibilidad que el ADR-002 justifica como críticos para reducir el acoplamiento	Media	Sincronizar los métodos de la fachada en el diagrama de clases con las responsabilidades descritas en los ADRs y el Nivel 3	Luciano	Abierto